

Advanced Topics and Tools in LLM Training

Distributed Training, Memory
Optimization, Evaluation & Frameworks

Presented by,
Prof. Karishma Bobde

Introduction to Large Language Model (LLM) Training

Modern LLMs such as GPT-style transformer models contain **billions to trillions of parameters**. Training such models:

- Requires **terabytes of data**
- Needs **high GPU memory (40GB–80GB per GPU)**
- Takes **weeks or months** on a single machine

Therefore, **distributed training** is used to:

- Reduce training time
- Overcome GPU memory limits
- Scale across multiple GPUs or multi-node clusters
- Improve hardware utilization

Introduction to Large Language Model (LLM) Training Continued....

- Distributed training mainly uses three strategies:
- Data Parallelism
- Model Parallelism
- Pipeline Parallelism
- Often, large systems combine all three
- (called **3D parallelism**).

Need for Distributed Training

- Single GPU memory is insufficient for large models.
- Training time becomes impractical without parallelism.
- Distributed systems allow scaling across GPUs and nodes.
- Enables training of billion/trillion parameter models.

Data Parallelism – Concept

- **Concept**
- Each GPU stores a **complete copy of the model**, but processes **different subsets of the dataset**.
- If there are N GPUs:
- Each GPU computes gradients independently.
- Gradients are synchronized before updating weights.

How It Works (Step-by-Step)

- Dataset is divided into mini-batches.
- Each GPU receives a different mini-batch.
- Each GPU performs:
 - Forward pass
 - Loss computation
 - Backward pass
- Gradients are averaged using All-Reduce.
- All GPUs update weights identically.

Communication Mechanism: All-Reduce

- All-Reduce ensures:
- Each GPU sends gradients to others.
- Gradients are averaged.
- Every GPU gets identical updated parameters.
- Used in:
PyTorch Distributed Data Parallel (DDP)

Advantages

Easy to implement

Works well when model fits into one GPU

Highly scalable with larger batch sizes

Strong support in modern frameworks

Limitations

Each GPU stores full model → memory intensive

Communication cost increases with GPU count

Limited by maximum batch size scaling

Model Parallelism – Concept

Concept

Instead of copying the full model to each GPU:
The model itself is divided across multiple GPUs.

Example:

GPU 1 $\rightarrow$ Layers 1–10

GPU 2 $\rightarrow$ Layers 11–20

How It Works

- Input is fed to GPU 1.
- GPU 1 computes first layers.
- Output activations sent to GPU 2.
- GPU 2 computes next layers.
- Backward pass flows in reverse direction.

Types of Model Parallelism

- **Layer-wise Parallelism**
- **Tensor Parallelism** (splitting large matrices)
- **Embedding Parallelism**

Advantages

- Enables training models larger than single GPU memory
Efficient for billion+ parameter models
Allows scaling of transformer layers

Limitations

- High inter-GPU communication
Sequential dependency slows execution
Harder to debug and implement

Model Parallelism – Challenges

- High inter-GPU communication latency.
- Complex implementation logic.
- Load balancing issues across devices.
- Debugging becomes more difficult.

Pipeline Parallelism – Concept

Concept

- Pipeline parallelism divides the model into **stages**, but processes multiple micro-batches in overlapping fashion.
- It combines:
- Layer partitioning
- Parallel execution

Example

- Suppose:
- GPU 1 $\rightarrow$ Layers 1–5
- GPU 2 $\rightarrow$ Layers 6–10
- Execution:
- Time Step 1:
GPU 1 $\rightarrow$ Batch 1
- Time Step 2:
GPU 1 $\rightarrow$ Batch 2
GPU 2 $\rightarrow$ Batch 1
- Time Step 3:
GPU 1 $\rightarrow$ Batch 3
GPU 2 $\rightarrow$ Batch 2
- This creates a **pipeline flow**.

Pipeline Parallelism – Advantages

Better GPU utilization

- ✓ Reduced idle time
- ✓ Higher throughput
- ✓ Efficient for deep transformer stacks

Pipeline Parallelism – Limitations

Pipeline scheduling complexity
Requires careful micro-batch tuning
Still communication heavy

Comparison Summary

Feature	Data Parallelism	Model Parallelism	Pipeline Parallelism
Model Copy	Full copy on each GPU	Split across GPUs	Split into stages
Memory Usage	High	Distributed	Distributed
Communication	Gradient sync	Activation transfer	Activation + scheduling
Best For	Medium models	Very large models	Deep models
Complexity	Low	Medium	High

Efficient Memory Management in LLMs

Training LLMs is extremely memory-intensive because of:

- Billions of parameters
- Long sequence lengths (context windows)
- Attention layers with quadratic complexity
- Storage of activations for backpropagation

For a transformer model, memory is consumed by:

- Model parameters
- Optimizer states (e.g., Adam uses $2\times$ parameters)
- Gradients
- Activations (largest during training)
- Among these, **self-attention layers dominate memory usage**, especially for long sequences.

Flash Attention – Concept

- Optimized attention computation algorithm.
- Avoids storing full attention matrix.
- Computes attention in memory-efficient blocks.
- IO-aware GPU optimization technique.

Flash Attention Solution

Flash Attention redesigns the computation to be:

- ✓ Block-wise
- ✓ Memory-efficient
- ✓ IO-aware

Flash Attention – Advantages

- Reduces memory complexity.
- Faster training speed.
- Enables longer sequence length training.
- Improves GPU utilization efficiency.

Evaluation of Fine-Tuned LLMs

Fine-tuning a Large Language Model (LLM) does not guarantee improved performance.

Therefore, **systematic evaluation and continuous monitoring** are essential to ensure:

- Generalization to unseen data
- Stability during training

Evaluation happens at two stages:

- During training (monitoring phase)
- After training (final evaluation phase)

Automatic Evaluation Metrics

Automatic metrics are quantitative and reproducible. They provide fast and objective measurement.

- Perplexity (PPL)
- BLEU (Bilingual Evaluation Understudy)
- ROUGE (Recall-Oriented Understudy for Gisting Evaluation)
- Accuracy & F1 Score

Automatic Evaluation Metrics- Perplexity (PPL)

Concept

Perplexity measures how well a language model predicts a sequence of words.

Lower perplexity $\rightarrow$ Better predictive performance.

Automatic Evaluation Metrics- BLEU (Bilingual Evaluation Understudy)

Used in:

- Machine translation
- Text generation

Idea

Measures overlap between:

- Generated text
- Reference (ground truth)

Automatic Evaluation Metrics- ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Used in:

- Text summarization

Measures:

- Overlap of n-grams
- Recall-based metric

Common variants:

- ROUGE-1 (unigrams)
- ROUGE-2 (bigrams)
- ROUGE-L (longest common subsequence)

Automatic Evaluation Metrics- Accuracy & F1 Score

- Used in:
- Classification tasks
- Sentiment analysis
- Intent detection

Comparison of Automatic Metrics

Metric	Best For	Measures	Limitation
Perplexity	Language Modeling	Prediction quality	Not semantic
BLEU	Translation	Precision overlap	Exact match bias
ROUGE	Summarization	Recall overlap	Surface-level
F1	Classification	Balance of P/R	Task-specific

Human Evaluation Metrics

Automatic metrics like BLEU, ROUGE, or Perplexity measure surface-level similarity or prediction probability

Human evaluation focuses on qualitative aspects that machines struggle to measure.

- Fluency – grammatical correctness.
- Coherence – logical flow of content.
- Relevance – task alignment.
- Bias and toxicity assessment.

Human Evaluation Metrics - Fluency

What It Measures:

- Grammatical correctness
- Sentence structure
- Natural phrasing
- Readability

Example:

- Prompt: *Explain gravity.*

Fluent response:

- Gravity is a fundamental force that attracts two objects with mass toward each other.

Non-fluent response:

- Gravity is force that pulling object toward another object because mass.

Human Evaluation Metrics - Coherence

What It Measures:

- Logical flow of ideas
- Context retention
- No contradictions
- Structured reasoning

Example of Poor Coherence:

- The Earth revolves around the Sun. It is flat in shape. This proves modern astronomy.
- Contradiction reduces coherence.

Human Evaluation Metrics - Relevance

- **What It Measures:**
 - Does the output directly answer the prompt?
 - Is it aligned with user intent?
 - Is unnecessary information added?

Example:

Prompt: *Give 3 advantages of AI.*

Poor relevance:

- AI is a branch of computer science developed in the 1950s...

Good relevance:

- Automation
- Improved decision-making
- Efficiency enhancement

Human Evaluation Metrics - Bias & Safety Checks

Critical for real-world deployment.

Human reviewers evaluate whether the model output contains:

A. Toxicity

- Offensive language
- Harmful statements

B. Hate Speech

- Targeting race, religion, gender, nationality

C. Political Bias

- Unbalanced political views
- Persuasive manipulation

Human Evaluation Metrics - Bias & Safety Checks

Continued...

D. Gender Bias

- Stereotypical assumptions
- Unequal treatment

E. Hallucination Risk

- Fabricated facts
- Confident but incorrect statements
- Safety evaluation is especially critical in research environments
- OpenAI
- Google DeepMind
- Human safety reviewers often:
- Follow structured evaluation rubrics
- Red-team models for harmful outputs
- Simulate adversarial prompts

Monitoring During Training

- Track training & validation loss curves.
- Monitor gradient norms.
- Early stopping to prevent overfitting.
- Experiment tracking tools for reproducibility.

Hugging Face Transformers

- Provides pre-trained transformer models.
- Supports GPT, BERT, T5 architectures.
- Easy fine-tuning with Trainer API.
- Large open-source model repository.

PEFT – Parameter Efficient Fine-Tuning

- Fine-tunes small subset of parameters.
- Methods: LoRA, Prefix Tuning, Adapters.
- Reduces GPU memory requirement.
- Faster and cost-effective training.

DeepSpeed Framework

- Developed for large-scale model training.
- ZeRO optimizer reduces memory redundancy.
- Supports pipeline & model parallelism.
- Mixed precision training support.

PyTorch Lightning

- High-level wrapper over PyTorch.
- Cleaner training loop structure.
- Built-in distributed training support.
- Logging and checkpointing features.

Conclusion

Distributed training enables scalable LLM training.

Flash Attention improves speed and memory efficiency.

Proper evaluation ensures model reliability.

Modern frameworks simplify implementation.



Thank You..!